

Dokumentasi Kode Simulasi MPC Konvensional Tanpa Halangan

1 Inisialisasi Lingkungan Kerja

Bagian ini digunakan untuk membersihkan lingkungan kerja MATLAB sebelum simulasi dijalankan agar tidak ada variabel atau plot sisa yang mengganggu.

```
1 clc; clear; close all;
```

- `clc;`: Membersihkan teks pada *Command Window*.
- `clear;`: Menghapus semua variabel dari *Workspace* memori.
- `close all;`: Menutup semua jendela figur (grafik) yang sedang terbuka.

2 Parameter Fisik Kapal

Blok ini mendefinisikan dimensi fisik, massa, dan parameter operasional kapal yang akan disimulasikan.

```
1 %% Parameter Kapal
2 L = 101.07;           % Panjang Kapal (m)
3 B = 14;               % Lebar Kapal (m)
4 T = 3.7;              % Draft Kapal (m)
5 m = 2423*1e3;         % Massa Kapal (kg)
6 u_0 = 15.4;           % Kecepatan surge Kapal (m/s)
7 C_B = 0.65;           % Koefisien Blok
8 x_G = 5.25;           % Pusat massa sumbu-x
9 A_delta = 5.7224;     % Luas rudder (m^2)
10 rho = 1024;           % Massa jenis air laut (kg/m^3)
11 r = 0.156*L;         % Kisaran jari-jari girasi
```

- `L`, `B`, `T`: Dimensi utama kapal yaitu panjang (*Length*), lebar (*Breadth*), dan sarat air (*Draft*) dalam satuan meter.
- `m`: Massa total kapal dalam kilogram (2.423×10^6 kg).
- `u_0`: Kecepatan jelajah arah maju (*surge*) diasumsikan konstan pada 15.4 m/s.
- `C_B`: Koefisien blok (*Block Coefficient*) yang merepresentasikan bentuk haluan/lambung kapal.
- `x_G`: Jarak pusat gravitasi dari titik tengah referensi kapal (*midship*).
- `A_delta`, `rho`, `r`: Luasan daun kemudi, massa jenis air laut standar, dan aproksimasi jari-jari girasi inersia.

3 Koefisien Hidrodinamika Non-Dimensional

Sistem dinamika kapal linier sangat bergantung pada gaya hidrodinamika (massa tambah dan redaman). Persamaan di bawah ini menggunakan pendekatan empiris (seperti metode Clarke dkk.) untuk menghitung turunan hidrodinamika dalam bentuk non-dimensional.

```
1 %% Koefisien Hidrodinamika
2 Y_v_dot = -(1 + 0.16*C_B*B/T - 5.1*(B/L)^2)*pi*(T/L)^2;
3 Y_r_dot = -(0.67*(B/L) - 0.0033*(B/T)^2)*pi*(T/L)^2;
4 N_v_dot = -(1.1*B/L - 0.041*B/T)*pi*(T/L)^2;
5 N_r_dot = -((1/12) + 0.017*C_B*B/T - 0.33*B/L)*pi*(T/L)^2;
6 Y_v = -(1 + 0.4*C_B*B/T)*pi*(T/L)^2;
7 Y_r = -(-0.5 + 2.2*B/L - 0.08*B/T)*pi*(T/L)^2;
8 N_v = -(0.5 + 2.4*T/L)*pi*(T/L)^2;
9 N_r = -(0.25 + 0.039*B/T - 0.56*B/L)*pi*(T/L)^2;
10 Y_delta = (rho*pi*A_delta)/(4*L*T);
11 N_delta = -0.5*Y_delta;
```

- Y_{v_dot} , Y_{r_dot} , N_{v_dot} , N_{r_dot} : Koefisien massa tambah (*added mass*) terhadap percepatan gerak *sway* dan *yaw*.
- Y_v , Y_r , N_v , N_r : Koefisien redaman hidrodinamik linier (*linear damping*) terhadap kecepatan gerak *sway* dan *yaw*.
- Y_{delta} , N_{delta} : Efektivitas gaya (*Sway force*) dan momen (*Yaw moment*) yang dihasilkan oleh perubahan sudut kemudi (*rudder*).

```
1 m_nd = 2*m/(rho*L^3);
2 x_G_nd = x_G/L;
3 I_z_nd = 1.2392*10^(-4);
4 u_0_nd = 1;
```

- Mengonversi parameter massa (m_{nd}), pusat gravitasi (x_{G_nd}), inersia (I_{z_nd}), dan kecepatan awal (u_{0_nd}) ke dalam bentuk tak-berdimensi (non-dimensional) berdasarkan pendekatan panjang kapal L dan kepadatan air laut ρ .

4 Pembentukan Model Gerak Kapal (State-Space Linier)

Blok ini merakit matriks *State-Space* yang akan memetakan dinamika *sway* dan *yaw*.

```
1 %% Model Gerak Kapal (Linier)
2 M = [m_nd-Y_v_dot, m_nd*x_G_nd-Y_r_dot ;
3      m_nd*x_G_nd - N_v_dot, I_z_nd - N_r_dot];
4 a11 = ((I_z_nd - N_r_dot)*Y_v - (m_nd*x_G_nd - Y_r_dot)*N_v)/det(M);
5 a12 = ((I_z_nd - N_r_dot)*(Y_r - m_nd*u_0_nd) - (m_nd*x_G_nd - Y_r_dot)*
6        (N_r - m_nd*x_G_nd*u_0_nd))/det(M);
7 a21 = ((m_nd - Y_v_dot)*N_v - (m_nd*x_G_nd - N_v_dot)*Y_v)/det(M);
8 a22 = ((m_nd - Y_v_dot)*(N_r - m_nd*x_G_nd*u_0_nd) - (m_nd*x_G_nd -
9        N_v_dot)*(Y_r - m_nd*u_0_nd))/det(M);
10 A_sys = [a11, a12; a21, a22];
11 B_sys = [0.01; 1];
```

- **M**: Matriks inersia yang mencakup massa kapal dan massa tambah air.
- **a11, a12, a21, a22**: Elemen-elemen pengali *damping* yang diselesaikan dengan invers matriks **M** menggunakan determinan ($\det(\mathbf{M})$).
- **A_sys**: Matriks sistem dinamika ordo-2 untuk *sway* (v) dan *yaw rate* (r).
- **B_sys**: Matriks input (efek dari *rudder*) terhadap kecepatan lateral dan rotasional.

Untuk menyimulasikan trajektori navigasi, sistem dikembangkan menjadi 5 *states* (v, r, x, y, ψ).

```

1 % State order: [v; r; x; y; psi]
2 A_lin = zeros(5,5);
3 A_lin(1:2, 1:2) = A_sys;
4 A_lin(4, 1) = 1;
5 A_lin(4, 5) = u_0_nd;
6 A_lin(5, 2) = 1;
7 B_lin = zeros(5,1);
8 B_lin(1:2) = B_sys;
9 D_affine = zeros(5,1);
10 D_affine(3) = u_0_nd;

```

- **A_lin**: Matriks sistem utama (5×5). Memasukkan matriks **A_sys** serta relasi kinematika (kecepatan terhadap posisi x, y dan *heading* ψ).
- **B_lin**: Matriks matriks kontrol kemudi terhadap 5 state utama.
- **D_affine**: Vektor tambahan untuk mengakomodasi gerak lurus konstan (*surge*) dalam matriks persamaan diferensial posisi sumbu x .

5 Konfigurasi LMPC Konvensional (Prediksi dan Diskritisasi)

MPC memerlukan sistem diskrit dan bobot *cost function*.

```

1 %% Setup LMPC Konvensional (Beda Hingga)
2 Tp = 60; % Horizon waktu prediksi (s)
3 Np = 60; % Jumlah langkah prediksi
4 dt_pred = Tp / Np; % Interval waktu prediksi diskrit
5 T_sim = 1; % Langkah waktu simulasi (s)
6 T_sim_total = 180; % Waktu simulasi total (s)

```

- **Tp, Np**: Parameter horizon prediksi. MPC akan melihat ke masa depan selama 60 detik yang dibagi menjadi 60 langkah.
- **dt_pred, T_sim, T_sim_total**: Skala interval langkah per prediksi dan total lama waktu jalannya simulasi.

```

1 % Diskritisasi Beda Hingga (Euler Forward) ke Non-Dimensional
2 dt_nd = dt_pred * (u_0 / L);
3 Ad = eye(5) + A_lin * dt_nd;
4 Bd = B_lin * dt_nd;
5 Dd = D_affine * dt_nd;

```

- `dt_nd`: Normalisasi langkah waktu menjadi non-dimensional (berdasarkan u_0 dan L).
- `Ad`, `Bd`, `Dd`: Merubah sistem dari domain waktu kontinu (*Continuous-time*) menjadi domain waktu diskrit (*Discrete-time*) menggunakan metode pendekatan *Euler Forward*.

```

1 % Bobot
2 Q = diag([50, 50, 50]);
3 Q_full = diag([0, 0, Q(1,1), Q(2,2), Q(3,3)]);
4 R = 100;

```

- `Q`, `Q_full`: Matriks bobot *penalty* untuk state kapal. Menghukum deviasi (*error*) posisi x, y , dan *heading* ψ agar kapal mematuhi referensi trajektori.
- `R`: Bobot penalti untuk aksi kontrol (agar sudut kemudi stabil dan tidak bergetar).

```

1 % Batasan state & input
2 r_limit = 0.0932;
3 r_limit_nd = r_limit * (L / u_0);
4 lb_state_single = [-Inf; -r_limit_nd; -Inf; -Inf; -Inf];
5 ub_state_single = [ Inf;  r_limit_nd;  Inf;  Inf;  Inf];
6
7 u_limit = deg2rad(35);
8 lb_input_single = -u_limit;
9 ub_input_single =  u_limit;
10
11 u_rate_limit = deg2rad(5) * dt_pred;
12 lb = [repmat(lb_state_single, Np+1, 1); repmat(lb_input_single, Np, 1)];
13 ub = [repmat(ub_state_single, Np+1, 1); repmat(ub_input_single, Np, 1)];

```

- `r_limit`, `r_limit_nd`: Pembatasan nilai gerak rotasi putar maksimum (*yaw rate*) agar kapal tidak terbalik saat berbelok tajam.
- `lb_state_single`, `ub_state_single`: Vektor batasan atas dan bawah untuk posisi state, dalam kode ini hanya membatasi State ke-2 (*yaw rate*).
- `u_limit`, `lb_input_single`, `ub_input_single`: Membatasi maksimal putaran fisik daun kemudi, umumnya pada sudut 35 derajat.
- `u_rate_limit`: Membatasi kelajuan maksimum perubahan daun kemudi (derajat per detik).
- `lb`, `ub`: Susunan matriks vertikal untuk batas optimasi QP untuk *states* dan *inputs* dari waktu $k = 1$ hingga horizon N_p .

6 Konstruksi Matriks Optimasi Quadratic Programming (QP)

Membentuk masalah MPC ke dalam bentuk standar *Quadprog*: $J = \frac{1}{2}Z^T H Z + f^T Z$ yang dibatasi oleh matriks kesetaraan (dinamika sistem) dan pertidaksamaan.

```

1 %% Membangun Matriks QP LMPC Konvensional
2 % 1. Matriks Hessian (H)
3 H_s = kron(eye(Np+1), Q_full);
4 H_u = kron(eye(Np), R);
5 H = blkdiag(H_s, H_u);
6 H = (H + H') / 2;
7 H = H + 1e-6 * eye(size(H));

```

- H_s , H_u : Membentuk blok diagonal matriks biaya state (Q) sepanjang waktu horizon dan biaya input (R). Fungsi `kron` melipatgandakan matriks dasar.
- H : Menggabungkan H_s dan H_u . Dilakukan simetrisasi $(H+H')/2$ dan penambahan faktor kondisioning numerik kecil $1e-6$ untuk memastikan matriks *Hessian* definit positif secara ketat.

```

1 % 2. Kendala Persamaan Dinamika Kapal:  $s(k+1) = A_d s(k) + B_d u(k) + D_d$ 
2 A_eq_dyn_s = kron(eye(Np+1), eye(5));
3 for i = 1:Np
4     A_eq_dyn_s(i*5+1:(i+1)*5, (i-1)*5+1:i*5) = -Ad;
5 end
6 A_eq_dyn_u = zeros(5*(Np+1), Np);
7 for i = 1:Np
8     A_eq_dyn_u(i*5+1:(i+1)*5, i) = -Bd;
9 end
10 A_eq = [A_eq_dyn_s, A_eq_dyn_u];
11 b_eq_base = zeros(5*(Np+1), 1);
12 for i = 1:Np
13     b_eq_base(i*5+1:(i+1)*5) = Dd;
14 end

```

- $A_{eq_dyn_s}$, $A_{eq_dyn_u}$: Membangun matriks koefisien di ruas kiri persamaan (*equality constraints*) sedemikian rupa sehingga memaksa pergerakan optimasi memenuhi kaidah $s(k+1) - A_d s(k) - B_d u(k) = D_d$.
- A_{eq} , b_{eq_base} : Matriks batas akhir dari persamaan kesetaraan. Variabel kondisi awal sistem (x_0) akan diinjeksi ke dalam b_{eq_base} pada siklus iterasi berjalan.

```

1 % 3. Kendala Pertidaksamaan Laju Input (delta u)
2 A_ineq_rate = zeros(Np, Np);
3 A_ineq_rate(1,1) = 1;
4 for i = 2:Np
5     A_ineq_rate(i, i) = 1;
6     A_ineq_rate(i, i-1) = -1;
7 end
8 A_ineq_u = [A_ineq_rate; -A_ineq_rate];
9 A_ineq = [zeros(2*Np, 5*(Np+1)), A_ineq_u];

```

- A_{ineq_rate} : Matriks differensial (*finite difference*) untuk mengawasi laju perubahan input (Δu) secara waktu berjalan.
- A_{ineq_u} , A_{ineq} : Matriks pertidaksamaan membatasi nilai minimum dan maksimum (*upper & lower rate limit*) dan disesuaikan dimensinya dengan total matriks Z (vektor parameter optimasi total).

7 Siklus Simulasi Utama LMPC

Blok ini merupakan putaran umpan-balik (*feedback loop*) di mana LMPC bekerja pada setiap langkah waktu, mengambil status aktual, menghitung optimasi kontrol, dan meneruskannya ke dinamika kapal.

```
1 %% Simulasi LMPC Konvensional
2 x0 = [0; 0; 0; 500; 0];
3 x0_nd = dimensional_to_nondimensional(x0, L, u_0);
4 h_ref = [0; 0; 0];
5 history_state_nd = [];
6 history_input = [];
7 input_initial = 0;
8 options = optimoptions('quadprog', 'Display', 'none', 'MaxIterations',
    2000);
9 fprintf('Memulai simulasi Linear MPC (Konvensional/Euler)...\n');
10 total_timer = tic;
```

- `x0`, `x0_nd`: Pendefinisian vektor kondisi awal kapal, kemudian dikonversi menjadi satuan non-dimensional.
- `h_ref`, `history_*`: Referensi target X, Y, ψ nol, serta alokasi memori pengumpul data hasil untuk *plotting*.
- `options`, `tic`: Pengaturan non-cetak (*silent execution*) untuk pustaka Quadprog dan pengaktifan *timer* (stopwatch) kecepatan komputasi.

Di bawah ini adalah siklus simulasinya:

```
1 for t = 0:T_sim:T_sim_total
2     % --- Update Vektor Referensi ---
3     ref_future = zeros(5, Np+1);
4     for k = 1:Np+1
5         t_predict_sekat = t + (k-1) * dt_pred;
6         ref_future(3, k) = (h_ref(1) + t_predict_sekat * u_0) / L;
7         ref_future(4, k) = h_ref(2) / L;
8         ref_future(5, k) = h_ref(3);
9     end
10    s_ref_flat = ref_future(:);
```

- `for t = ...`: Siklus utama simulasi waktu dari 0 hingga detik ke 180.
- `ref_future`: Mengalkulasi titik referensi (bayangan target) sepanjang horizon waktu untuk referensi posisi sumbu-X yang terus bergerak maju (karena kecepatan surganya u_0).

```
1 % --- Update Vektor Gradien (f) ---
2 f_s = -H_s * s_ref_flat;
3 f_u = zeros(Np, 1);
4 f = [f_s; f_u];
5
6 % --- Update Batasan Kondisi Awal ---
7 b_eq = b_eq_base;
8 b_eq(1:5) = x0_nd;
```

- `f_s`, `f`: Membentuk vektor matriks linier (f) untuk fungsi biaya optimasi berdasarkan jarak error (referensi sistem dikalikan parameter $Cost$ matriks H_s).
- `b_eq(1:5)`: Mengunci (*hard constraint*) nilai 5 elemen pertama sama persis dengan x_0 (posisi riil terkini).

```

1 % --- Update Batasan Laju Input Awal ---
2 b_ineq_limit1 = u_rate_limit * ones(Np, 1);
3 b_ineq_limit1(1) = u_rate_limit + input_initial; % Batas atas u_0
4
5 b_ineq_limit2 = u_rate_limit * ones(Np, 1);
6 b_ineq_limit2(1) = u_rate_limit - input_initial; % Batas bawah u_0
7
8 b_ineq = [b_ineq_limit1; b_ineq_limit2];

```

- `b_ineq_limit`: Pembaruan parameter pertidaksamaan, menghitung ruang kosong sudut yang bisa ditempuh oleh kendali kemudi, dengan mengacu ke posisi sudut kemudi terakhir yang diberikan (`input_initial`).

```

1 % --- Optimasi QP ---
2 [z_opt, ~, exitflag] = quadprog(H, f, A_ineq, b_ineq, A_eq, b_eq, lb,
3 ub, [], options);
4
5 if exitflag ~= 1
6     warning('Quadprog gagal konvergen pada t = %d. Exitflag: %d', t,
7 exitflag);
8 end

```

- `quadprog(...)`: *Solver* optimasi QP dari MATLAB yang menyelesaikan sekuen keputusan masukan terbaik untuk sepanjang N_p langkah ke depan.
- `exitflag`: Menghasilkan peringatan jika masalah iterasi tidak mendapatkan solusi (infeasible/unbounded).

```

1 u_apply = z_opt(5*(Np+1) + 1);
2 input_initial = u_apply;
3 history_state_nd = [history_state_nd, x0_nd];
4 history_input = [history_input, u_apply];

```

- `u_apply`: Kaidah MPC *Receding Horizon*, di mana dari total N_p aksi kontrol masa depan yang dihasilkan mesin QP, hanya masukan pada indeks waktu pertama saja yang akan diinjeksi ke aktuator kapal.
- `history_*`: Perekaman nilai status aktual simulasi.

```

1 % Simulasi dinamika linier kapal (Runge-Kutta 4)
2 dt_nd_sim = T_sim * u_0 / L;
3 k1 = linear_ship_dynamics(x0_nd, u_apply, A_lin, B_lin, D_affine);
4 k2 = linear_ship_dynamics(x0_nd + 0.5*dt_nd_sim*k1, u_apply, A_lin,
5 B_lin, D_affine);
6 k3 = linear_ship_dynamics(x0_nd + 0.5*dt_nd_sim*k2, u_apply, A_lin,
7 B_lin, D_affine);

```

```

6     k4 = linear_ship_dynamics(x0_nd + dt_nd_sim*k3, u_apply, A_lin,
    B_lin, D_affine);
7     x0_nd = x0_nd + (dt_nd_sim/6) * (k1 + 2*k2 + 2*k3 + k4);
8
9     if mod(t, 20) == 0
10         disp(['Waktu Simulasi: ', num2str(t), ' s']);
11     end
12 end
13 waktu_total = toc(total_timer);

```

- k1, k2, k3, k4: Menggunakan metode *Runge-Kutta 4th Order* (RK4) untuk memperkirakan perpindahan aktual kapal menuju detik berikutnya dengan presisi sangat tinggi dibandingkan iterasi biasa.
- toc: Mematikan hitungan pewaktu setelah *loop* selesai.

Konversi kembali ke skala asli (dimensional):

```

1 history_state_dim = zeros(size(history_state_nd));
2 for i = 1:size(history_state_nd, 2)
3     history_state_dim(:, i) = nondimensional_to_dimensional(
4         history_state_nd(:, i), L, u_0);
5 end

```

8 Visualisasi dan Validasi (Plotting)

Menampilkan hasil berupa trajektori jelajah dan validasi operasional sistem batas (*constraints*).

```

1 %% Plot Hasil (Lengkap dengan Verifikasi Batasan)
2 figure('Name', 'Hasil Simulasi LMPC Konvensional');
3 time_vector = 0:T_sim:T_sim_total;
4
5 % Plot 1: Trajektori
6 subplot(2,1,1);
7 plot(history_state_dim(3,:), history_state_dim(4,:), 'b-', 'LineWidth',
8     2); hold on;
9 plot([min(history_state_dim(3,:)) max(history_state_dim(3,:))], [0 0], '
10     r--', 'LineWidth', 1.5);
11 xlabel('Posisi X'); ylabel('Posisi Y');
12 title('Trajektori Kapal vs Referensi');
13 legend('Kapal', 'Referensi'); grid on; axis equal;
14
15 % Plot 2: Heading
16 subplot(2,1,2);
17 plot(time_vector, rad2deg(history_state_dim(5,:)), 'b-', 'LineWidth', 2)
18 ; hold on;
19 y_ref_heading = rad2deg(h_ref(3)) * ones(size(time_vector));
20 plot(time_vector, y_ref_heading, 'r--', 'LineWidth', 1.5);
21 xlabel('Waktu (s)'); ylabel('Heading (derajat)');
22 title('Sudut Haluan (Yaw)');
23 legend('Actual', 'Reff'); grid on;

```

- subplot(2,1,1): Grafik membandingkan pergerakan riil posisi $x - y$ (biru) dibandingkan rute sasaran (garis putus merah).

- `subplot(2,1,2)`: Menginspeksi sudut orientasi (*heading / yaw*) kapal sepanjang waktu simulasi.

```

1 %% Plot Verifikasi Batasan (Constraints)
2 figure('Name', 'Verifikasi Batasan (Constraints)');
3 % Plot 1: Yaw Rate
4 subplot(3,1,1);
5 ...
6 % Plot 2: Rudder Input
7 subplot(3,1,2);
8 ...
9 % Plot 3: Rudder Rate
10 subplot(3,1,3);
11 u_actual_deg = rad2deg(history_input);
12 u_rate_calc = [0, diff(u_actual_deg)] / T_sim;
13 ...

```

- `figure(Verifikasi Batasan)`: Rangkaian 3 grafik turunan performa penggerak aktuator.
- `Yaw Rate`: Membuktikan bahwa nilai kecepatan rotasi (deg/s) yang diraih tak pernah melanggar batas yang telah diset dalam `r_limit`.
- `Rudder Input`: Memvalidasi sudut belok tidak melampaui batasan keras $\pm 35^\circ$.
- `Rudder Rate`: Menguji apakah differensial selisih perintah kendali, `diff(u_actual_deg)`, mematuhi batasan dinamika motor kemudi.

9 Fungsi Pendukung Lokal

Blok terakhir ini berisi 3 sub-fungsi yang dipanggil dalam siklus LMPC dan RK4.

```

1 %% Fungsi Dinamika Linier Kapal
2 function s_dot = linear_ship_dynamics(s, u, A_lin, B_lin, D_affine)
3     s_dot = A_lin * s + B_lin * u + D_affine;
4 end

```

- `linear_ship_dynamics`: Memetakan hukum matriks laju $\dot{x}(t) = Ax(t) + Bu(t) + D$ sebagai input solver Runge-Kutta 4.

```

1 %% Fungsi Konversi Nondimensional
2 function x_nd = dimensional_to_nondimensional(x_dim, L, u_0)
3     x_nd = [x_dim(1)/u_0; x_dim(2)*L/u_0; x_dim(3)/L; x_dim(4)/L; x_dim(5)];
4 end
5 function x_dim = nondimensional_to_dimensional(x_nd, L, u_0)
6     x_dim = [x_nd(1)*u_0; x_nd(2)*u_0/L; x_nd(3)*L; x_nd(4)*L; x_nd(5)];
7 end

```

- `dimensional_to_nondimensional`: Normalisasi panjang meter dan kelajuan dengan rasio konstanta L dan kecepatan u_0 . Kecepatan menjadi proporsi murni tanpa metrik real.
- `nondimensional_to_dimensional`: Membalik nilai yang diproses algoritma MPC kembali ke dunia nyata (satuan SI) untuk dicetak pada grafik.